ODONTOCLINIC

Documentación Técnica del Sistema

Proyecto de Seminario

Ingeniería en Sistemas

Universidad Mariano Gálvez de Guatemala

Estudiante: [Completar información]

Docente: [Completar información]

Sede: [Completar información]

Fecha: 5 de octubre de 2026

Control del documento

Campo	Valor
Nombre del documento	Documentación Técnica de OdontoClinic
Sistema	OdontoClinic
Versión del documento	1.0
Fecha de elaboración	5 de octubre de 2026
Estado	Auditoría técnica del código fuente
Versiones declaradas del sistema	Frontend 0.0.0; backend 0.0.1

Este documento describe el estado verificable del repositorio auditado. Las recomendaciones aparecen separadas de los controles implementados y no se presentan como capacidades actuales.

Tabla de contenido

Control del documento 2
Tabla de contenido 3
1 Introducción 6
2 Objetivo de la documentación 6
3 Alcance del sistema 6
3.1 Funcionalidades implementadas 6
3.2 Funcionalidades parciales o con desacople 6
3.3 Elementos encontrados sin uso operativo demostrado 6
4 Descripción general del sistema 7
5 Arquitectura del sistema 8
5.1 Responsabilidades por capa 8
6 Stack tecnológico 8
7 Estructura del proyecto 9
8 Arquitectura del Frontend 10
8.1 Rutas 10
8.2 Servicios y comunicación 10
8.3 Estado y almacenamiento 10
8.4 Manejo de errores frontend 10
9 Arquitectura del Backend 11
9.1 Inicio y configuración HTTP 11
10 Módulos funcionales 11
10.1 Autenticación 11
10.2 Dashboard 11
10.3 Agenda y citas 11
10.4 Pacientes 11
10.5 Historia clínica 12
10.6 Reportes 12
10.7 Usuarios y perfil 12
11 Autenticación y autorización 12
11.1 Mecanismo de autenticación 12
11.2 Protección efectiva 13
11.3 Expiración, refresh y logout 13
12 Roles y permisos 13
13 API REST 14
13.1 Raíz y autenticación 14

13.2 Usuarios 14
13.3 Pacientes 14
13.4 Citas 14
13.5 Historia clínica 15
13.6 Cuerpos principales 15
14 Base de datos 16
14.1 Enums 16
15 Diccionario de datos 16
15.1 Usuario 16
15.2 Paciente 17
15.3 Cita 17
15.4 HistoriaClinica 17
16 Modelo entidad relación 18
17 Flujos principales del sistema 18
17.1 Inicio de sesión 18
17.2 Alta de paciente 19
17.3 Programación de cita 19
17.4 Consulta de historia 19
17.5 Generación de reporte 19
18 Seguridad 19
18.1 Controles implementados 19
18.2 Riesgos identificados 19
19 Configuración del sistema 20
20 Variables de entorno 20
21 Instalación y ejecución local 21
21.1 Prerrequisitos verificables 21
21.2 Backend 21
21.3 Frontend 21
21.4 Build 21
22 Scripts disponibles 21
22.1 Frontend 21
22.2 Backend 21
23 Despliegue 22
24 Manejo de errores 22
25 Validaciones 22
26 Pruebas 23

27 Dependencias principales 23
28 Consideraciones de mantenimiento 23
29 Limitaciones técnicas identificadas 24
30 Recomendaciones técnicas 24
31 Información pendiente de documentación 25
32 Conclusiones técnicas 25
Anexo A Trazabilidad de evidencia 26
Anexo B Estado de la auditoría 26

En Microsoft Word, seleccione la tabla y use Actualizar campo para recalcular títulos y páginas después de editar el documento.

1 Introducción

OdontoClinic es una aplicación web para gestionar usuarios de la clínica, pacientes, citas, consulta de historial basado en citas y generación de reportes. El repositorio contiene un frontend Angular y una API NestJS que persiste datos mediante Prisma en PostgreSQL. La aplicación ofrece autenticación con JWT, aunque la protección del backend se aplica únicamente a dos rutas del módulo de autenticación.

La auditoría encontró una separación clara entre interfaz, API y persistencia, junto con diferencias entre los controles visibles en el frontend y la autorización aplicada por el servidor. Estas diferencias son relevantes para operar y mantener el sistema de forma segura.

2 Objetivo de la documentación

El objetivo es permitir que un desarrollador comprenda la estructura, las dependencias, los contratos HTTP, el modelo de datos, la autenticación, la configuración y el procedimiento de ejecución que pueden demostrarse con el código actual. El documento también registra limitaciones técnicas y recomendaciones sin atribuir al sistema funciones no implementadas.

3 Alcance del sistema

3.1 Funcionalidades implementadas

- Inicio de sesión con correo y contraseña, emisión de un JWT y almacenamiento de la respuesta en localStorage.
- Dashboard con métricas derivadas de pacientes y citas de una fecha seleccionada.
- Agenda con vistas diaria, semanal y mensual; alta, edición, cambio de estado y eliminación de citas.
- Alta, consulta, edición, filtrado, paginación y eliminación condicionada de pacientes.
- Consulta de perfil de paciente e historial construido con citas asociadas.
- Reportes de pacientes, citas e historial clínico agregados en el navegador y exportables a CSV, XLSX y PDF.
- Gestión de usuarios desde la interfaz, con roles ADMIN, DOCTOR y RECEPCIONISTA.
- API CRUD para usuarios, pacientes, citas e historias clínicas, además de registro, login, perfil y actualización autenticada.

3.2 Funcionalidades parciales o con desacople

- El frontend denomina historia clínica a una vista de citas y notas; no consume el CRUD backend de HistoriaClinica ni muestra diagnóstico, tratamiento u observaciones persistidos en esa entidad.
- El servicio de reportes contiene rutas HTTP hipotéticas dentro de una rama desactivada. El backend no implementa un controlador de reportes; la ruta activa agrega datos en el cliente.
- Los parámetros date, search y pacienteId se envían desde el frontend, pero los controladores backend no los reciben ni procesan. La aplicación vuelve a filtrar en el cliente.
- Existe un RolesGuard y un decorador Roles, pero ningún controlador los aplica.
- El campo refreshToken existe en el esquema, pero el frontend lo utiliza como identificador del doctor asignado a una recepcionista. No existe una operación de renovación de JWT.

3.3 Elementos encontrados sin uso operativo demostrado

- Las clases de entidad TypeScript de Usuario, Paciente, Cita e HistoriaClinica están vacías.

- frontend/src/app/pacientes/temp.service.ts es una copia incompleta que no aparece importada por la aplicación compilada.
- backend/dev.db tiene tamaño cero; el esquema activo configura PostgreSQL y no referencia ese archivo.
- GET / y sus pruebas conservan la respuesta de plantilla Hello World!.

4 Descripción general del sistema

El navegador arranca una aplicación Angular basada en componentes standalone. Las rutas funcionales se protegen con authGuard, que comprueba si existe un access_token en localStorage. Los servicios de dominio usan HttpClient contra una URL de API codificada en el frontend. Un interceptor agrega el encabezado Authorization Bearer cuando encuentra un token.

NestJS recibe las solicitudes en controladores por recurso y delega en servicios que ejecutan operaciones Prisma. Prisma obtiene la conexión PostgreSQL desde DATABASE_URL. La API no establece un prefijo global, por lo que los paths documentados parten de la raíz del servidor.

5 Arquitectura del sistema

La arquitectura observada es cliente servidor con separación de frontend, API y capa de persistencia. El frontend concentra parte de la lógica de alcance de datos: filtra citas y pacientes por el usuario almacenado, resuelve nombres de pacientes con solicitudes adicionales y agrega reportes. El backend ofrece CRUD directo mediante Prisma.

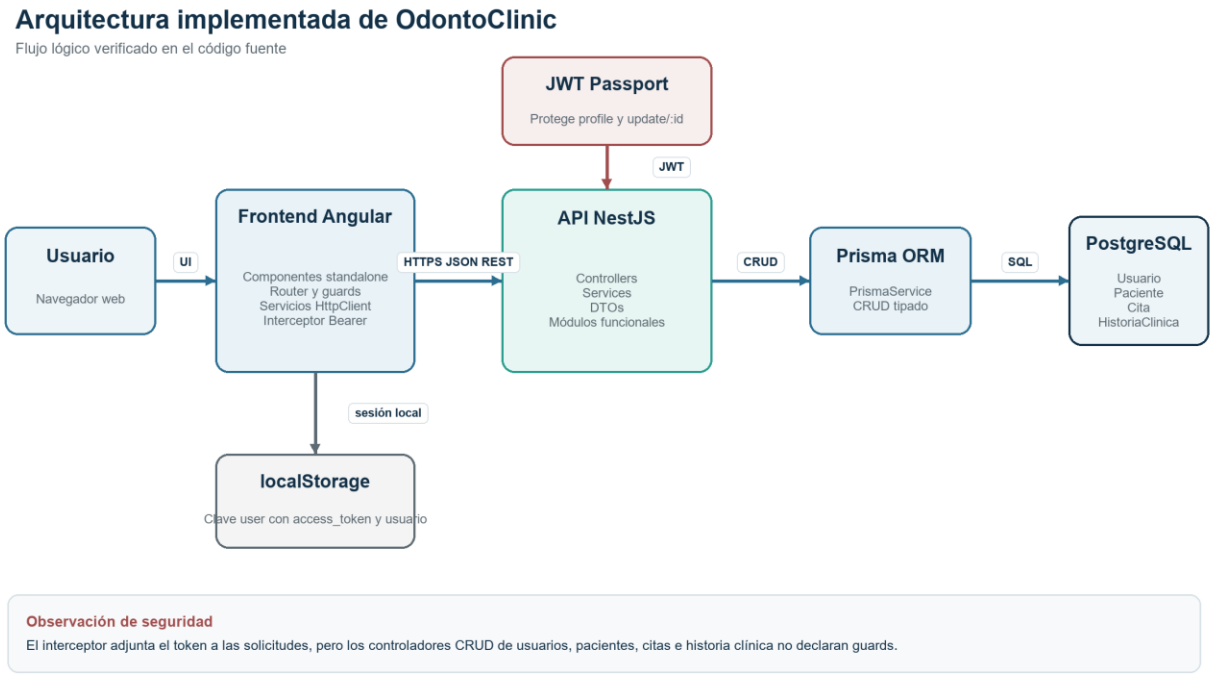


Figura 1. Arquitectura lógica reconstruida desde el código fuente.

5.1 Responsabilidades por capa

Capa	Responsabilidad verificada	Evidencia principal
Presentación	Vistas, formularios, filtros, navegación y exportación de reportes.	frontend/src/app
Cliente HTTP	Mapeo de modelos, transformación de fechas, filtros locales y consumo REST.	Servicios Angular por módulo
API	Rutas HTTP, autenticación, servicios CRUD y respuestas de NestJS.	backend/src
Acceso a datos	PrismaClient global, conexión y operaciones sobre modelos.	backend/src/prisma
Persistencia	Tablas, enums, restricciones y relaciones PostgreSQL.	backend/prisma/schema.prisma y migración

6 Stack tecnológico

Tecnología	Versión comprobada	Uso	Evidencia
Angular	19.2.25	SPA y componentes standalone	frontend/package-lock.json

Tecnología	Versión comprobada	Uso	Evidencia
Angular CLI	19.2.27	Build, serve y pruebas	frontend/package-lock.json
TypeScript frontend	5.7.3	Código del cliente	frontend/package-lock.json
Bootstrap	5.3.8	Estilos y modales	frontend/package-lock.json y index.html
RxJS	7.8.2	Flujos HTTP y estado asíncrono	package-lock de ambos componentes
ExcelJS	4.4.0	Exportación XLSX	frontend/package-lock.json
jsPDF	3.0.4	Exportación PDF	frontend/package-lock.json
jsPDF AutoTable	5.0.8	Tablas en PDF	frontend/package-lock.json
NestJS	11.1.6	API HTTP modular	backend/package-lock.json
Prisma Client	6.17.1	ORM y acceso a datos	backend/package-lock.json
PostgreSQL	No fijada	Motor definido por el datasource	backend/prisma/schema.prisma
bcrypt	6.0.0	Hash y comparación de contraseñas	backend/package-lock.json
Passport JWT	4.0.1	Extracción y validación Bearer	backend/package-lock.json
Jest	30.1.3	Pruebas backend	backend/package-lock.json
Karma	6.4.4	Pruebas frontend	frontend/package-lock.json

7 Estructura del proyecto

El repositorio mantiene frontend y backend como proyectos npm independientes. El package.json raíz solo declara @nestjs/config y no coordina scripts de ambos componentes.

```
odonto_clinic/
├─ frontend/
│  ├─ src/app/                componentes, rutas, guards y servicios
│  ├─ src/assets/             recursos SVG
│  ├─ angular.json            build, test y deploy gh-pages
│  └─ package.json            scripts y dependencias del cliente
├─ backend/
│  ├─ src/auth/               login, JWT, DTOs y guards
│  ├─ src/{usuarios,pacientes,citas,historia-clinica}/
│  ├─ src/prisma/             módulo y servicio Prisma
│  ├─ prisma/                 schema y migración inicial
│  ├─ test/                   prueba e2e de la ruta raíz
│  └─ package.json            scripts y dependencias de la API
└─ package.json               dependencia raíz sin scripts
```

8 Arquitectura del Frontend

El frontend usa bootstrapApplication y componentes standalone. app.config.ts registra el router, HttpClient con AuthInterceptor y coalescencia de eventos. Las rutas login, dashboard, agenda, pacientes, historia clínica, reportes, configuración y perfil se declaran en app.routes.ts.

8.1 Rutas

Ruta	Componente	Carga	Protección
/login	LoginComponent	Lazy	loginGuard evita mostrarla si existe token
/dashboard	DashboardComponent	Directa	authGuard
/agenda	AgendaComponent	Directa	authGuard
/pacientes	PacientesComponent	Directa	authGuard
/pacientes/:id	PacientePerfilComponent	Lazy	authGuard
/historia-clinica	HistoriaClinicaComponent	Directa	authGuard
/reportes	ReportesComponent	Directa	authGuard
/configuracion	ConfiguracionComponent	Directa	authGuard; no guard de rol
/perfil	PerfilComponent	Lazy	authGuard
**	Redirección	-	Envía a /dashboard

8.2 Servicios y comunicación

Servicio	Responsabilidad	API activa
AuthService	Login, logout, token y usuario actual	https://odonto-clinic.onrender.com
AgendaService	CRUD de citas, fechas, filtros y nombres de pacientes	Misma URL
PacientesService	CRUD, mapeo de fechas y alcance por usuario	Misma URL
HistoriaClinicaService	Combina paciente con citas filtradas	Misma URL
UsuariosService	Gestión de usuarios	Misma URL
ReportesService	Agrega datos en el cliente	No usa /reportes mientras useClientAggregation sea true

8.3 Estado y almacenamiento

La aplicación usa signals y computed para estado de interfaz. La autenticación se guarda como JSON bajo la clave user de localStorage. El interceptor lee access_token de esa estructura. Los almacenes mock dentalpro_pacientes_v1 y dentalpro_usuarios_v1 permanecen en servicios, pero useMock está configurado en false.

8.4 Manejo de errores frontend

Los componentes gestionan errores mediante callbacks de subscribe y mensajes de interfaz. Algunos servicios sustituyen errores por arreglos vacíos o valores undefined mediante catchError.

HistoriaClinicaService aplica timeout de 10 segundos y, ante cualquier error, devuelve un paciente vacío con citas vacías, por lo que puede ocultar la causa original a la vista.

9 Arquitectura del Backend

AppModule importa UsuariosModule, PacientesModule, CitasModule, HistoriaClinicaModule y AuthModule. PrismaModule es global y expone PrismaService. Cada módulo CRUD sigue Controller a Service a PrismaService. No hay capa repository adicional ni filtros globales de excepciones registrados.

Módulo	Controlador	Servicio	Persistencia
Auth	AuthController	AuthService	Usuario mediante Prisma
Usuarios	UsuariosController	UsuariosService	Usuario
Pacientes	PacientesController	PacientesService	Paciente
Citas	CitasController	CitasService	Cita
Historia clínica	HistoriaClinicaController	HistoriaClinicaService	HistoriaClinica
Prisma	-	PrismaService	Conexión y desconexión del cliente

9.1 Inicio y configuración HTTP

- NestFactory crea AppModule y escucha PORT o 3000.
- CORS permite http://localhost:4200, https://yorchriv.github.io y https://odonto-clinic.onrender.com, con credenciales y métodos CRUD.
- No se registra ValidationPipe global, prefijo global, Swagger, Helmet, rate limiting ni filtro global de excepciones.

10 Módulos funcionales

10.1 Autenticación

Valida usuario activo, compara la contraseña con bcrypt y firma un JWT. Expone registro, login, perfil y actualización autenticada.

10.2 Dashboard

Combina AgendaService y PacientesService para mostrar conteos y accesos rápidos; no tiene endpoint propio.

10.3 Agenda y citas

Gestiona citas, estados y periodos. El backend ofrece CRUD de Cita; el frontend filtra por usuario y fecha.

10.4 Pacientes

Gestiona datos personales, estado y alergias. El backend incluye creador, citas e historias al consultar pacientes.

10.5 Historia clínica

El backend persiste HistoriaClinica, pero la vista actual reconstruye un historial de citas y notas sin consumir esa entidad.

10.6 Reportes

Genera reportes en el cliente y exporta CSV, XLSX y PDF. No existe módulo backend de reportes.

10.7 Usuarios y perfil

La configuración permite gestionar cuentas y asignar una recepcionista a un doctor usando refreshToken. El perfil actualiza datos y contraseña mediante UsuariosService.

11 Autenticación y autorización



Figura 2. Secuencia exitosa de inicio de sesión.

11.1 Mecanismo de autenticación

1. LoginComponent valida email y contraseña con Angular Validators.
2. AuthService envía POST /auth/login.
3. AuthService del backend busca Usuario por email, verifica activo y ejecuta bcrypt.compare.
4. JwtService firma un payload con email, sub, rol, nombre y apellido; el login establece expiresIn en 24h.
5. El frontend guarda la respuesta completa en localStorage y navega a /dashboard.
6. AuthInterceptor adjunta el token como Bearer a solicitudes posteriores.

11.2 Protección efectiva

JwtAuthGuard está aplicado únicamente a GET /auth/profile y PUT /auth/update/:id. La segunda ruta permite actualizar al propio usuario o a cualquier usuario cuando req.user.rol es ADMIN. Los controladores de usuarios, pacientes, citas e historia clínica no declaran UseGuards.

11.3 Expiración, refresh y logout

El token emitido por login expira en 24 horas. AuthModule también declara signOptions de 7 días, pero login reemplaza ese valor al firmar. No existe endpoint de refresh. El campo refreshToken se devuelve al cliente, pero no participa en renovación; el frontend lo interpreta como id de doctor. Logout solo elimina la clave user de localStorage.

12 Roles y permisos

Los roles persistidos son ADMIN, DOCTOR y RECEPCIONISTA. La matriz distingue comportamiento de interfaz y control del servidor. Una restricción aplicada solo en el frontend no constituye autorización backend.

Funcionalidad	ADMIN	DOCTOR	RECEPCIONISTA	Enforcement backend
Ver navegación de usuarios	Visible	Oculta	Oculta	Ninguno en /configuracion; es ruta autenticada solamente
Crear usuario desde UI	Visible	No visible	No visible	POST /auth/register es público
Editar usuario desde configuración	Sí	Propio si entra por URL	Propio si entra por URL	PATCH /usuarios/:id no protegido
Eliminar usuario	Visible	Oculto	Oculto	DELETE /usuarios/:id no protegido
Consultar pacientes	Filtra por creador propio	Filtra por creador propio	Doctor asignado o propia cuenta	GET /pacientes devuelve todos
Gestionar citas	Agenda propia	Agenda propia	Agenda del doctor guardado en refreshToken	CRUD /citas no protegido
Eliminar cita	Permitido UI	Permitido UI	Bloqueado UI	DELETE /citas/:id no protegido
Eliminar paciente	Permitido si no tiene citas	Permitido si no tiene citas	Bloqueado UI	DELETE /pacientes/:id no verifica rol ni citas
Actualizar /auth/update/:id	Cualquier id	Solo propio id	Solo propio id	JwtAuthGuard y comprobación explícita

13 API REST

La API expone 25 rutas. Salvo las dos rutas marcadas con JWT, los controladores no declaran autenticación. Los DTOs CRUD sin decoradores funcionan como formas TypeScript, no como validación runtime demostrada.

13.1 Raíz y autenticación

Método	Endpoint	Descripción	Autenticación	Entrada	Respuesta observada
GET	/	Respuesta de plantilla	No	Ninguna	Hello World!
POST	/auth/register	Registra usuario y hashea contraseña	No	RegisterDto	Usuario sin password
POST	/auth/login	Valida credenciales y emite token	No	LoginDto	access_token, user y message
GET	/auth/profile	Obtiene identidad seleccionada	JWT	Bearer token	id, email, nombre, apellido y rol
PUT	/auth/update/:id	Actualiza usuario	JWT	id y UpdateUserDto	Usuario sin password

13.2 Usuarios

Método	Endpoint	Descripción	Autenticación	Entrada	Respuesta
POST	/usuarios	Crea usuario	No declarada	Body Create DTO	Registro creado por Prisma
GET	/usuarios	Lista usuario	No declarada	Sin parámetros efectivos	Arreglo de registros
GET	/usuarios/:id	Obtiene usuario por id	No declarada	id en ruta	Registro o null
PATCH	/usuarios/:id	Actualiza usuario	No declarada	id y Update DTO	Registro actualizado
DELETE	/usuarios/:id	Elimina usuario	No declarada	id en ruta	Registro eliminado

13.3 Pacientes

Método	Endpoint	Descripción	Autenticación	Entrada	Respuesta
POST	/pacientes	Crea paciente	No declarada	Body Create DTO	Registro creado por Prisma
GET	/pacientes	Lista paciente	No declarada	Sin parámetros efectivos	Arreglo de registros
GET	/pacientes/:id	Obtiene paciente por id	No declarada	id en ruta	Registro o null
PATCH	/pacientes/:id	Actualiza paciente	No declarada	id y Update DTO	Registro actualizado
DELETE	/pacientes/:id	Elimina paciente	No declarada	id en ruta	Registro eliminado

13.4 Citas

Método	Endpoint	Descripción	Autenticación	Entrada	Respuesta
POST	/citas	Crea cita	No declarada	Body Create DTO	Registro creado por Prisma

Método	Endpoint	Descripción	Autenticación	Entrada	Respuesta
GET	/citas	Lista cita	No declarada	Sin parámetros efectivos	Arreglo de registros
GET	/citas/:id	Obtiene cita por id	No declarada	id en ruta	Registro o null
PATCH	/citas/:id	Actualiza cita	No declarada	id y Update DTO	Registro actualizado
DELETE	/citas/:id	Elimina cita	No declarada	id en ruta	Registro eliminado

13.5 Historia clínica

Método	Endpoint	Descripción	Autenticación	Entrada	Respuesta
POST	/historia-clinica	Crea historia clínica	No declarada	Body Create DTO	Registro creado por Prisma
GET	/historia-clinica	Lista historia clínica	No declarada	Sin parámetros efectivos	Arreglo de registros
GET	/historia-clinica/:id	Obtiene historia clínica por id	No declarada	id en ruta	Registro o null
PATCH	/historia-clinica/:id	Actualiza historia clínica	No declarada	id y Update DTO	Registro actualizado
DELETE	/historia-clinica/:id	Elimina historia clínica	No declarada	id en ruta	Registro eliminado

13.6 Cuerpos principales

DTO	Campos
RegisterDto	email, password, nombre, apellido, rol, refreshToken opcional
LoginDto	email, password
CreateUsuarioDto	nombre, apellido, email, password, rol, refreshToken, lastLogin, activo
CreatePacienteDto	nombres, apellidos, dpi, telefono, email, direccion, fechaNacimiento, estado, alergias, creadoPorId
CreateCitaDto	pacienteId, usuarioId, fecha, hora, motivo, estado, notas
CreateHistoriaClinicaDto	citaId, pacienteId, descripcion, diagnostico, tratamiento, observaciones

14 Base de datos

Prisma configura un datasource PostgreSQL mediante DATABASE_URL. La migración inicial crea tres enums y cuatro tablas. Las claves foráneas usan ON DELETE RESTRICT y ON UPDATE CASCADE. No se declaran índices adicionales fuera de las restricciones únicas de email, dpi y citaId.

Modelo	Propósito verificable	Relaciones
Usuario	Cuenta, rol, credenciales y estado	Crea pacientes y atiende citas
Paciente	Datos personales y clínicos básicos	Creador, citas e historias
Cita	Fecha, hora, motivo, estado y notas	Paciente, usuario e historia opcional
HistoriaClinica	Descripción, diagnóstico, tratamiento y observaciones	Una cita única y un paciente

14.1 Enums

Enum	Valores
Rol	ADMIN, DOCTOR, RECEPCIONISTA
EstadoPaciente	ACTIVO, INACTIVO
EstadoCita	CONFIRMADA, PENDIENTE, NUEVA, FINALIZADA, CANCELADA, DISPONIBLE

15 Diccionario de datos

15.1 Usuario

Campo	Tipo	Obligatorio	Clave o regla	Descripción técnica
id	Int	Sí	PK, autoincrement	Identificador del usuario.
nombre	String	Sí	-	Nombre.
apellido	String	Sí	-	Apellido.
email	String	Sí	Único	Correo usado por login.
password	String	Sí	-	Hash en el flujo de registro; no hay restricción DB de formato.
rol	Rol	Sí	Default RECEPCIONISTA	Rol persistido.
refreshToken	String	No	-	Nombrado como refresh token; usado por frontend como id de doctor.
lastLogin	DateTime	No	-	Campo disponible; no se actualiza en el código auditado.
activo	Boolean	Sí	Default true	Habilita o bloquea login y validación JWT.
creadoEn	DateTime	Sí	Default now	Fecha de creación.
actualizadoEn	DateTime	Sí	updatedAt	Fecha actualizada por Prisma.

15.2 Paciente

Campo	Tipo	Obligatorio	Clave o regla	Descripción técnica
id	Int	Sí	PK, autoincrement	Identificador.
nombres	String	Sí	-	Nombres del paciente.
apellidos	String	Sí	-	Apellidos.
dpi	String	No	Único	Identificador documental opcional.
telefono	String	Sí	-	Teléfono.
email	String	No	-	Correo opcional.
direccion	String	No	-	Dirección opcional.
fechaNacimiento	DateTime	No	-	Fecha de nacimiento.
estado	EstadoPaciente	Sí	Default ACTIVO	Estado del paciente.
alergias	String	No	-	Texto libre sobre alergias.
creadoPorId	Int	Sí	FK Usuario	Usuario creador.
creadoEn	DateTime	Sí	Default now	Fecha de creación.
actualizadoEn	DateTime	Sí	updatedAt	Última actualización.

15.3 Cita

Campo	Tipo	Obligatorio	Clave o regla	Descripción técnica
id	Int	Sí	PK, autoincrement	Identificador.
pacienteId	Int	Sí	FK Paciente	Paciente asociado.
usuarioId	Int	Sí	FK Usuario	Usuario asociado a la agenda.
fecha	DateTime	Sí	-	Fecha y hora construidas por el frontend.
hora	String	Sí	-	Hora separada en HH:MM desde el frontend.
motivo	String	Sí	-	Motivo de consulta.
estado	EstadoCita	Sí	Default NUEVA	Estado operativo.
notas	String	No	-	Notas libres.
creadoEn	DateTime	Sí	Default now	Fecha de creación.
actualizadoEn	DateTime	Sí	updatedAt	Última actualización.

15.4 HistoriaClinica

Campo	Tipo	Obligatorio	Clave o regla	Descripción técnica
id	Int	Sí	PK, autoincrement	Identificador.
citaId	Int	Sí	FK Cita, único	Vincula como máximo una historia a cada cita.
pacienteId	Int	Sí	FK Paciente	Paciente asociado.

Campo	Tipo	Obligatorio	Clave o regla	Descripción técnica
descripcion	String	No	-	Descripción clínica libre.
diagnostico	String	No	-	Diagnóstico libre.
tratamiento	String	No	-	Tratamiento libre.
observaciones	String	No	-	Observaciones libres.
creadoEn	DateTime	Sí	Default now	Fecha de creación.
actualizadoEn	DateTime	Sí	updatedAt	Última actualización.

16 Modelo entidad relación

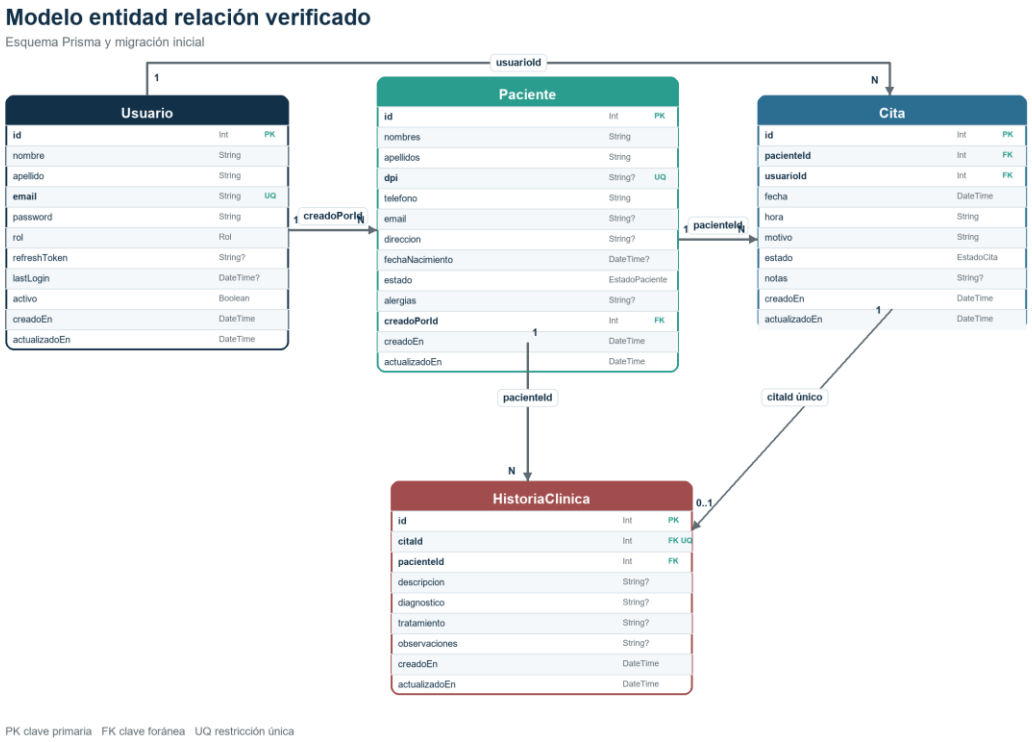


Figura 3. Modelo entidad relación derivado de schema.prisma y la migración inicial.

La unicidad de HistoriaClinica.citaId establece una relación Cita 1 a HistoriaClinica 0..1. Paciente y Usuario mantienen relaciones uno a muchos con citas; Usuario también mantiene una relación uno a muchos con pacientes creados.

17 Flujos principales del sistema

17.1 Inicio de sesión

Actor: usuario. El formulario Angular envía credenciales a POST /auth/login. AuthService consulta Usuario, verifica activo y password, firma un token y devuelve identidad. El frontend guarda la respuesta y navega al dashboard.

17.2 Alta de paciente

Actor: usuario autenticado en la interfaz. PacientesComponent valida campos HTML, PacientesService obtiene el usuario local, transforma fecha y creadoPorId y envía POST /pacientes. PacientesController delega a PacientesService, que ejecuta prisma.paciente.create. La API devuelve el registro creado.

17.3 Programación de cita

AgendaComponent exige seleccionar un paciente existente. AgendaService determina usuarioId desde el usuario actual o desde refreshToken para recepcionistas, convierte la fecha y envía POST /citas. La API persiste el registro con Prisma y el frontend consulta el paciente para completar su nombre.

17.4 Consulta de historia

HistoriaClinicaComponent selecciona un paciente. HistoriaClinicaService obtiene /pacientes/:id y /citas, filtra las citas por paciente y doctor en el cliente, normaliza fecha y hora y agrupa los resultados. Este flujo no consulta /historia-clinica.

17.5 Generación de reporte

ReportesComponent construye filtros y llama ReportesService. Con useClientAggregation en true, el servicio consulta PacientesService, AgendaService o HistoriaClinicaService, consolida filas en memoria y permite descargarlas como CSV, XLSX o PDF.

18 Seguridad

18.1 Controles implementados

Control	Implementación comprobada	Alcance
Hash de contraseña	bcrypt con costo 10 en registro y actualizaciones que pasan por AuthService o UsuariosService.update	No se aplica en UsuariosService.create
JWT	Bearer token, expiración comprobada y usuario activo en JwtStrategy	Solo rutas que usan JwtAuthGuard
Guard frontend	Comprueba existencia de access_token	No valida firma ni expiración
CORS	Lista explícita de tres orígenes y métodos	Configuración global Nest
Validadores declarativos	class-validator en LoginDto, RegisterDto y UpdateUserDto	Sin ValidationPipe global demostrado
Restricciones DB	Claves, foráneas, enums y únicos	PostgreSQL mediante migración
CSV	Prefija celdas que empiezan con = + - @	Exportación CSV de reportes

18.2 Riesgos identificados

Prioridad	Hallazgo	Evidencia técnica	Impacto
Crítica	CRUD clínico y de usuarios sin guards backend	Controllers no usan UseGuards	Acceso directo posible sin autorización del servidor

Prioridad	Hallazgo	Evidencia técnica	Impacto
Alta	GET /usuarios devuelve registros completos	prisma.usuario.findMany sin select	Puede exponer hashes y refreshToken
Alta	POST /usuarios no hashea password y registra el DTO	UsuariosService.create	Contraseña en claro y posible registro en logs
Alta	Fallback literal para JWT en JwtStrategy y configuración distinta en JwtModule	jwt.strategy.ts y auth.module.ts	Configuración inconsistente y secreto predecible si falta la variable
Alta	Autorización clínica aplicada en el cliente	Filtros por creadoPorId y usuarioId en servicios Angular	Un cliente alterno puede omitir filtros
Media	Token completo en localStorage	AuthService	Exposición ante XSS
Media	Sin ValidationPipe global	main.ts	Decoradores DTO no se ejecutan automáticamente
Media	refreshToken usado como id de doctor	Servicios Angular y formulario de usuarios	Confusión semántica y bloqueo de un refresh real
Media	UpdateUserDto declara ASISTENTE en lugar de RECEPCIONISTA	auth/dto/update-user.dto.ts	Inconsistencia si se activa validación
Baja	Errores clínicos convertidos en datos vacíos	HistoriaClinicaService catchError	Dificulta diagnóstico operativo

19 Configuración del sistema

Archivo	Configuración relevante
frontend/angular.json	Build production/development, assets, Bootstrap CSS, Karma y builder angular-cli-ghpages.
frontend/tsconfig.json	TypeScript estricto, target ES2022 y strictTemplates.
backend/nest-cli.json	sourceRoot src y borrado de dist antes de compilar.
backend/tsconfig.json	NodeNext, target ES2023, decoradores y salida dist.
backend/src/main.ts	CORS y PORT con default 3000.
backend/prisma/schema.prisma	PostgreSQL, DATABASE_URL, modelos y enums.

No se encontraron archivos .env ni .env.example en el repositorio. No se copiaron valores sensibles al documento.

20 Variables de entorno

Variable	Componente	Propósito	Obligatoria según código
DATABASE_URL	Backend Prisma	Cadena de conexión PostgreSQL	Sí para conectar y migrar
JWT_SECRET	Backend Auth	Firma y validación de JWT	Debe serlo; JwtStrategy tiene un fallback literal inseguro
PORT	Backend	Puerto de escucha	No; default 3000

21 Instalación y ejecución local

21.1 Prerrequisitos verificables

- Node.js y npm; el repositorio no fija versión mediante engines, .nvmrc o archivo equivalente.
- Una instancia PostgreSQL accesible mediante DATABASE_URL.
- Un navegador compatible para el frontend; las pruebas Karma están configuradas para Chrome.

21.2 Backend

```
cd backend
npm install
npx prisma generate
npx prisma migrate dev --name init
npm run start:dev
```

Los comandos Prisma aparecen comentados en schema.prisma. Antes de iniciar se deben proporcionar DATABASE_URL y JWT_SECRET sin registrar sus valores en Git.

21.3 Frontend

```
cd frontend
npm install
npm start
```

El servidor de desarrollo Angular usa normalmente http://localhost:4200, origen permitido por CORS. Sin embargo, los servicios activos apuntan a https://odonto-clinic.onrender.com; las URLs locales aparecen comentadas. El repositorio no implementa una configuración environment para alternar endpoints sin editar código.

21.4 Build

```
cd backend && npm run build
cd frontend && npm run build
```

22 Scripts disponibles

22.1 Frontend

Script	Comando	Uso
start	ng serve	Servidor de desarrollo
build	ng build	Build de producción por defecto
watch	ng build --watch --configuration development	Compilación observada
test	ng test	Karma/Jasmine

22.2 Backend

Script	Comando	Uso

Script	Comando	Uso
build	nest build	Compilar API
start	nest start	Inicio normal
start:dev	nest start --watch	Desarrollo
start:debug	nest start --debug --watch	Depuración
start:prod	node dist/main	Ejecutar build
lint	eslint ... --fix	Lint con cambios automáticos
test	jest	Unitarias
test:e2e	jest --config ./test/jest-e2e.json	E2E
test:cov	jest --coverage	Cobertura
format	prettier --write ...	Formateo

23 Despliegue

angular.json configura angular-cli-ghpages como builder de deploy. El frontend usa una URL onrender.com para la API y main.ts autoriza un origen github.io. Estas referencias demuestran intención o uso de esos servicios, pero el repositorio no contiene workflows, Dockerfiles, render.yaml, configuración de dominio, comandos de release ni documentación suficiente para reconstruir un despliegue completo y repetible.

El README backend conserva recomendaciones genéricas del template NestJS, incluida una opción Mau, pero no prueba que OdontoClinic se despliegue con ese servicio.

24 Manejo de errores

NestJS usa excepciones estándar en autenticación: UnauthorizedException y ConflictException. Los CRUD dejan que Prisma y el manejador predeterminado de Nest transformen errores. No se observaron filtros personalizados ni un contrato uniforme de error.

El frontend muestra mensajes específicos en login y formularios, pero varias comparaciones buscan err.message locales que no necesariamente coinciden con respuestas HTTP. catchError se usa para degradar algunas consultas a arreglos vacíos, y los componentes registran ciertos errores con console.error.

25 Validaciones

Capa	Validación observada	Limitación
Formularios Angular	required, email, minlength y selección de paciente	Solo cliente
DTO auth	IsEmail, IsString, IsNotEmpty, MinLength, IsEnum e IsOptional	Sin ValidationPipe global
DTO CRUD	Tipos TypeScript y enums locales	Sin decoradores class-validator
Servicios	Duplicado de email, usuario activo, password y propiedad de update auth	No cubre CRUD público
Prisma/PostgreSQL	Tipos, no nulos, únicos, claves foráneas y enums	No expresa todas las reglas de interfaz

26 Pruebas

Se encontraron 23 archivos .spec.ts y una prueba e2e. La mayoría de las pruebas solo verifican que un componente, controlador o servicio pueda crearse. La prueba unitaria de AppController y la e2e verifican Hello World!.

Validación ejecutada	Resultado el 5 de octubre de 2026	Detalle
Frontend npm run build	Aprobada	Generó dist/odonto_clinik; mostró advertencias CommonJS y selectores CSS de Bootstrap.
Frontend npm test -- --watch=false	No ejecutada	Karma inició, pero no encontró Chrome en la ruta configurada.
Backend npm run build	No ejecutada	El comando nest no estaba disponible porque las dependencias backend no estaban instaladas.
Backend npm test -- --runInBand	No ejecutada	El comando jest no estaba disponible por la misma condición.

Estos resultados no demuestran corrección funcional completa. El build frontend sí confirma que el grafo importado de la aplicación compila en el estado auditado.

27 Dependencias principales

Componente	Dependencias relevantes	Finalidad
Frontend	@angular/*, rxjs, zone.js	Framework, enrutamiento, formularios y reactividad
Frontend UI	bootstrap	Estilos y comportamiento de modales
Frontend reportes	exceljs, file-saver, jspdf, jspdf-autotable	Exportaciones
Backend	@nestjs/common, core, platform-express, config	API y configuración
Backend auth	@nestjs/jwt, passport, passport-jwt, bcrypt	JWT y contraseñas
Backend datos	@prisma/client, prisma	ORM, cliente y migraciones
Validación	class-validator, class-transformer	Decoradores DTO; falta activar pipe global

28 Consideraciones de mantenimiento

- Mantener sincronizados schema.prisma, migraciones, DTOs y modelos frontend; hoy existen enums inconsistentes.
- Centralizar la URL de API para evitar editar varios servicios al cambiar de ambiente.
- Tratar refreshToken como deuda semántica: actualmente almacena una relación de asignación clínica.
- Recordar que los reportes dependen de múltiples consultas del cliente y del filtrado local.
- No asumir que HistoriaClinica de base de datos aparece en la vista; son implementaciones desacopladas.
- Los README son plantillas de Angular y NestJS y no describen por completo la configuración real.

- Antes de actualizar dependencias de reportes, repetir el build porque Angular reporta optimizaciones CommonJS.

29 Limitaciones técnicas identificadas

Limitación	Justificación
Autorización no centralizada	Los filtros de alcance y bloqueos por rol están en Angular; los CRUD backend son accesibles sin guards.
Consultas no filtradas en servidor	Los controladores listan todos los registros y no consumen query params.
Historia clínica incompleta de extremo a extremo	La entidad existe, pero el frontend usa citas.
Configuración por ambiente ausente	Las URLs están codificadas y no hay environments ni .env.example.
Pruebas superficiales	La mayoría solo comprueba creación; no cubren reglas, auth ni CRUD.
Despliegue no reproducible	No hay CI/CD ni manifest de backend.
Contrato de errores no uniforme	Mezcla excepciones Nest, Prisma y valores vacíos en frontend.

30 Recomendaciones técnicas

Prioridad	Recomendación	Resultado esperado
1	Aplicar JwtAuthGuard global o por controlador y RolesGuard con una matriz de permisos definida en backend.	Autorización independiente del cliente
1	Eliminar POST /usuarios o hacerlo equivalente al registro seguro; excluir password y refreshToken de respuestas y logs.	Reducir exposición de credenciales
1	Exigir JWT_SECRET sin fallback y rotar cualquier valor usado fuera del gestor de secretos.	Firma coherente y no predecible
1	Activar ValidationPipe con whitelist, transform y forbidNonWhitelisted; añadir decoradores a DTOs CRUD.	Validación runtime consistente
2	Crear una relación explícita doctor recepcionista en Prisma y reservar refreshToken para autenticación o eliminarlo.	Modelo semántico claro
2	Mover filtros de usuario, fecha, búsqueda y paciente a consultas Prisma autorizadas.	Menor exposición y mejor rendimiento
2	Integrar el módulo HistoriaClinica en el frontend o retirar el CRUD hasta definir el flujo clínico.	Consistencia funcional
2	Centralizar API base URL en environments y agregar .env.example sin secretos.	Ejecución local y despliegue repetibles
3	Agregar pruebas de AuthService, guards, permisos, DTOs, CRUD, filtros y escenarios e2e con base aislada.	Cobertura de reglas críticas
3	Añadir CI que instale con npm ci, genere Prisma, compile y ejecute pruebas.	Validación automática del repositorio
3	Agregar logging estructurado sin datos sensibles y un filtro uniforme de excepciones.	Diagnóstico operativo seguro
3	Evaluar cookies HttpOnly o controles XSS reforzados para el token según el modelo de despliegue.	Menor exposición del token

31 Información pendiente de documentación

- Credenciales y procedimientos de provisión de los ambientes, que no deben almacenarse en el repositorio.
- Versión operativa de Node.js y PostgreSQL aprobada para producción.
- Procedimiento exacto de despliegue de frontend y backend, dominios, responsables y rollback.
- Política institucional de roles y permisos que deba aplicar el backend.
- Definición funcional de historia clínica, diagnóstico y tratamiento en la interfaz.
- Política de respaldo, retención, cifrado y recuperación de la base de datos.
- Requisitos legales y de privacidad aplicables a datos personales y clínicos.
- Datos académicos de portada: estudiante, docente, sede y otros campos institucionales.

32 Conclusiones técnicas

OdontoClinic tiene una base funcional verificable: un SPA Angular compila, consume una API NestJS y persiste cuatro modelos relacionados mediante Prisma y PostgreSQL. La estructura por módulos facilita ubicar responsabilidades y las funciones principales de pacientes, citas, usuarios y reportes están presentes en la interfaz.

El principal límite técnico es la diferencia entre la seguridad percibida en el frontend y la protección efectiva del backend. La API aplica JWT solo en dos rutas, mientras que los CRUD restantes aceptan solicitudes sin guards. También existe desacople entre la entidad HistoriaClinica y la vista, y el campo refreshToken cumple una función distinta de su nombre.

La prioridad de mantenimiento debe ser reforzar autorización y validación en el servidor, evitar exposición de credenciales, alinear el modelo de asignación clínica y cubrir esas reglas con pruebas automatizadas. Después de esas acciones, la configuración por ambiente y el despliegue reproducible serían los siguientes pasos para estabilizar el proyecto.

Anexo A Trazabilidad de evidencia

Afirmación	Archivos que la demuestran
Angular standalone con router e interceptor	frontend/src/main.ts; app/app.config.ts; app/app.routes.ts
API NestJS modular	backend/src/app.module.ts y módulos de dominio
PostgreSQL con Prisma	backend/prisma/schema.prisma; migration.sql
JWT Bearer	backend/src/auth/auth.module.ts; strategies/jwt.strategy.ts; guards/jwt-auth.guard.ts
Solo dos rutas protegidas	backend/src/auth/auth.controller.ts; búsqueda de UseGuards en backend/src
Roles persistidos	schema.prisma enum Rol; frontend configuracion.service.ts
Reportes agregados en cliente	frontend/src/app/reportes/reportes.service.ts
Historia frontend basada en citas	frontend/src/app/historia-clinica/historia-clinica.service.ts
URL API codificada	auth.service.ts y servicios de dominio frontend
Variables de entorno	schema.prisma; auth.module.ts; jwt.strategy.ts; main.ts
Pruebas existentes	archivos *.spec.ts; backend/test/app.e2e-spec.ts; package.json
Deploy gh-pages configurado	frontend/angular.json

Anexo B Estado de la auditoría

Elemento	Cantidad o estado
Archivos del repositorio excluyendo dependencias y artefactos	134
Archivos TypeScript backend	50
Archivos TypeScript frontend	36
Componentes frontend no test	10
Servicios frontend no test	7
Controladores backend	6
Módulos backend	7
Servicios backend	7
Endpoints HTTP	25
Modelos Prisma	4
Archivos spec	23
Build frontend	Aprobado con advertencias
Build y pruebas backend	No ejecutables sin dependencias instaladas